
ServeFlow

Manual do Sistema e Arquitetura

Guia completo de estudo: tecnologias, motivações, exemplos práticos e uso do sistema

Componente	Tecnologia	Plataforma
Frontend	React 18 + TailwindCSS + Axios	Vercel
Backend	Spring Boot 3.4.3 / Java 21	Render
Banco	PostgreSQL 15 + Flyway	Supabase
Auth	JWT (JJWT 0.12.3) + BCrypt	—
Real-time	WebSocket + STOMP	—
E-mail	Gmail SMTP + STARTTLS	—
CEP	ViaCEP REST API	—

Versão 2.0 | Maio 2026 | 207 classes Java | 26 migrations Flyway
Spring Boot 3.4.3 / Java 21 / Clean Architecture + DDD

1. Introducao

O ServeFlow e um sistema de gestao para pequenos restaurantes, desenvolvido com foco em praticidade, seguranca e escalabilidade. Cobre o ciclo completo de operacao: usuarios, cardapios, pedidos, estoque, caixa e financeiro — com visualizacao em tempo real via WebSocket.

Este documento serve como guia de estudo e referencia tecnica: para cada tecnologia e padrao adotado, explica o **motivo da escolha**, o **proposito dentro do sistema** e um **exemplo pratico concreto**.

2. Visao Geral da Arquitetura

O sistema e dividido em tres camadas principais hospedadas em plataformas distintas:

Camada	Plataforma	Tecnologia Principal
Frontend	Vercel	React 18, TailwindCSS, Axios, Zustand
Backend	Render	Spring Boot 3.4.3, Java 21, Maven
Banco	Supabase	PostgreSQL 15, Connection Pooling, Flyway
E-mail	Gmail	SMTP :587, STARTTLS
CEP	ViaCEP	REST API publica — consulta de endereco

3. Frontend — Por que cada tecnologia?

3.1 React 18

O **que e** Biblioteca JavaScript para construcao de interfaces declarativas baseadas em componentes reutilizaveis.

Por que usamos?

React resolve o problema de manter a interface sincronizada com dados que mudam constantemente (status de pedidos, estoque, caixa) sem recarregar a pagina inteira. Re-renderiza apenas os componentes afetados — eficiente e responsivo.

Para que serve no ServeFlow: Cada tela (login, pedidos, KDS, caixa, estoque, financeiro) e um conjunto de componentes React reutilizaveis.

Exemplo pratico:

O garcom seleciona um produto e a quantidade total do pedido atualiza instantaneamente.
Sem React: seria necessario recarregar a pagina inteira ou manipulacao manual do DOM.
Com React: apenas o componente 'TotalPedido' re-renderiza em milissegundos.

3.2 React Router DOM

O que é: Biblioteca de roteamento client-side — gerencia URLs sem recarregar o servidor (SPA navigation).

Por que usamos?

Em uma SPA, a navegacao entre telas nao pode usar reload de pagina — isso seria lento e perderia o estado da aplicacao. React Router intercepta cliques em links e troca apenas o componente exibido, mantendo a URL sincronizada.

Exemplo pratico:

Gerente clica em 'Estoque' -> URL muda para /stock/consolidated
Nenhuma requisicao e feita ao servidor para essa navegacao.
O browser apenas troca o componente React exibido — rapido como trocar de aba.
Rotas protegidas por perfil:
/orders -> GARCOM, GERENTE, ADMIN
/kds -> COZINHEIRO, GERENTE, ADMIN
/cashier -> CAIXA, ADMIN
/financial -> GERENTE, ADMIN

3.3 TailwindCSS

O que é: Framework CSS utility-first — estilizacao diretamente no JSX com classes padronizadas, sem arquivos CSS separados.

Por que usamos?

CSS tradicional exige criar arquivos .css para cada componente, gerando duplicacao e inconsistencias visuais. TailwindCSS centraliza um sistema de design tokens (cores, espacamentos, tipografia) aplicados como classes — garantindo consistencia.

Exemplo pratico:

```
<button className="bg-green-500 hover:bg-green-600 text-white  
px-6 py-3 rounded-lg font-semibold  
transition-colors duration-200">  
Confirmar Pedido  
</button>
```

Sem escrever um unico arquivo .css: cor, hover, padding, bordas e animacao.
Classes 'md:grid-cols-2 lg:grid-cols-4' adaptam o layout para tablet e desktop.

3.4 Axios (com Interceptors)

O que é: Cliente HTTP para JavaScript com suporte a promises e interceptors — codigo que executa automaticamente antes/depois de cada requisicao.

Por que usamos?

O fetch nativo do browser nao tem interceptors. Sem Axios, o tratamento de token expirado precisaria ser repetido em cada chamada HTTP do sistema. O interceptor centraliza essa logica: detecta 401, renova o token e reexecuta a requisicao — completamente transparente para o usuario.

Exemplo pratico:

```
// Configurado uma unica vez no projeto:
axios.interceptors.response.use(
  response => response,
  async error => {
    if (error.response.status === 401) {
      const newToken = await refreshToken(); // chama /auth/refresh
      error.config.headers.Authorization = `Bearer ${newToken}`;
      return axios(error.config); // reexecuta a requisicao
    }
    return Promise.reject(error);
  }
);

Gerente consulta financeiro -> token de 15min expira -> Axios renova -> sem novo login.
```

3.5 Zustand / Context API

O que é Zustand e uma biblioteca leve de estado global para React. Evita o 'prop drilling' (passar dados por múltiplos componentes).

Por que usamos?

Em React, dados passados de pai para filho via props se tornam instáveis quando muitos níveis de componentes precisam do mesmo dado. Zustand resolve com uma store global simples e performática — mais leve que Redux.

Exemplo pratico:

```
// Qualquer componente acessa diretamente, sem prop drilling:
const { user } = useAuthStore();

// Menu lateral exibe/oculta itens por role:
if (user.role === 'ADMIN') show(<ConfigMenu />)
if (user.role === 'COZINHEIRO') show(<KdsMenu />)

Sem Zustand: o 'user' precisaria descer por 5 níveis de componentes via props.
```

4. Backend — Por que cada tecnologia e padrao?

4.1 Spring Boot 3.4.3

O que é Framework Java que configura automaticamente todo o ecossistema Spring — servidor embutido (Tomcat), JPA, segurança, WebSocket — com mínimo de boilerplate (convention over configuration).

Por que usamos?

Spring Boot elimina o 'configuration hell' do Spring tradicional. Com apenas a anotação `@SpringBootApplication`, o framework detecta as dependências no classpath e configura tudo automaticamente — DataSource, EntityManager, filtros de segurança, WebSocket broker.

Exemplo pratico:

application.yml — basta isso para conectar ao banco:

spring:

datasource:

url: jdbc:postgresql://host:5432/serveflow_db

username: postgres

password: \${DB_PASSWORD}

Sem Spring Boot: configuracao manual de DataSource, EntityManagerFactory, TransactionManager, HikariCP, Hibernate... centenas de linhas de XML.

Com Spring Boot: zero configuracao adicional.

Onde encontrar: ServeflowApplication.java (classe principal).

4.2 Clean Architecture

O que é Padrao arquitetural que separa regras de negocio (interno: Domain + Service) dos detalhes de implementacao (externo: HTTP, banco, frameworks).

Por que usamos?

Sem separacao de camadas, o codigo de negocio fica misturado com infraestrutura — impossivel testar sem banco de dados real, dificil de manter quando o framework muda. Clean Architecture garante que a logica de negocio nao depende de frameworks ou banco.

Exemplo pratico:

Camada externa (Infraestrutura):

OrderController -> recebe HTTP, valida DTO, chama OrderService

OrderEntity -> mapeamento JPA (@Entity), conhece banco

Camada interna (Dominio — sem dependencia de framework):

Order -> regras de negocio puras

OrderService -> orquestra casos de uso

OrderRepository -> interface (nao implementacao!)

Se migrarmos de PostgreSQL para MongoDB:

OrderEntity muda -> OrderService, Order, OrderController NAO SAO TOCADOS.

4.3 DDD — Domain-Driven Design

O que é: Abordagem onde o código reflete o vocabulário e as regras do domínio de negócio. Os objetos de domínio representam conceitos reais do restaurante.

Por que usamos?

Sem DDD, o código fica orientado a tabelas de banco (UserTable, OrderRow) em vez de conceitos de negócio (User, Order). DDD melhora a comunicação entre devs e facilita entender regras de negócio diretamente no código-fonte.

Exemplo prático:

```
// Com DDD – a regra 'só confirma se CREATED' vive no domínio:
public class Order {
    public void confirm() {
        if (this.status != OrderStatus.CREATED) {
            throw new BusinessException("Pedido já processado");
        }
        this.status = OrderStatus.CONFIRMED;
    }
}

// Sem DDD – a regra estaria espalhada em 3 lugares diferentes:
if (order.getStatus().equals("CREATED")) { ... } // no controller
if (status == 0) { ... } // no service
if (order.status.equals("CREATED")) { ... } // no repository
```

4.4 JWT — JSON Web Token

O que é: Padrão de token compacto e autocontido que representa claims (afirmações) sobre um usuário, assinado criptograficamente com HMAC-SHA256.

Por que usamos?

Autenticação tradicional baseada em sessão exige que o servidor armazene e consulte sessões a cada requisição — não escala horizontalmente. JWT é stateless: o token carrega os dados do usuário e é validado por criptografia, sem consultar banco de dados. Se escalarmos para 3 instâncias do backend, todas validam o mesmo token.

Exemplo prático:

```
Estrutura do JWT (3 partes separadas por ponto):
Header: eyJhbGciOiJIUzI1NiJ9
-> {"alg": "HS256"}
Payload: eyJ1c2VySWQiOiJEsInVzZXJuYXZlIjoiam9hbyIsInVjbGUiOiJHRVJFTlRFIn0
-> {"userId":1, "username":"joao", "role":"GERENTE", "exp":1748295600}
Assinatura: validada com JWT_SECRET (HMAC-SHA256)
-> garante que ninguém adulterou o payload
Backend sabe que é João, gerente, sem consultar a tabela users.
```

Onde encontrar: `config/JwtService.java`

4.5 BCrypt Password Encoder

O que é: Algoritmo de hash adaptativo para senhas com salt automático, projetado para ser intencionalmente lento e resistente a ataques de força bruta.

Por que usamos?

MD5 e SHA-1 são rápidos — um atacante pode testar bilhões de senhas por segundo com GPU. BCrypt é intencionalmente lento (custo configurável) e adiciona um salt único por senha. Mesmo que o banco de dados seja comprometido, as senhas são irreversíveis.

Exemplo prático:

```
Senha original: "admin123"
Hash BCrypt: "$2a$10$N9qo8uL0ickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lih57"
Dois usuarios com a mesma senha 'admin123' terao hashes DIFERENTES (salt aleatorio).
Impossivel reverter: nao existe funcao inversa para BCrypt.
// Uso no UserService:
String hash = passwordEncoder.encode("admin123"); // ao cadastrar
boolean ok = passwordEncoder.matches("admin123", hash); // ao logar
```

Onde encontrar: `config/SecurityConfig.java` -> `bean passwordEncoder()`

4.6 JwtFilter — OncePerRequestFilter

O que é: Filtro Spring Security que executa exatamente uma vez por requisição HTTP, validando o token JWT antes de qualquer controller ser chamado.

Por que usamos?

Precisamos que toda requisição autenticada seja validada antes de chegar ao controller. `OncePerRequestFilter` garante execução única mesmo em chains de filtros complexas. Sem este filtro, qualquer usuário poderia acessar qualquer endpoint sem token.

Exemplo prático:

```
GET /api/stock/items
Authorization: Bearer eyJ...

JwtFilter executa (antes do StockController):
1. Extrai 'eyJ...' do header Authorization
2. JwtService.validateToken() -> OK (assinatura valida, não expirado)
3. Extrai username='joao', role='GERENTE'
4. SecurityContextHolder.setAuthentication(...)

Spring Security verifica RBAC:
/stock/** requer GERENTE ou ADMIN -> OK -> chega no StockController
Se o token for inválido: retorna 401 Unauthorized imediatamente.
```

Onde encontrar: `config/JwtFilter.java`

4.7 CORS — Cross-Origin Resource Sharing

O que é: Mecanismo de segurança dos browsers que bloqueia requisições HTTP entre domínios diferentes, a menos que o servidor explicitamente autorize.

Por que usamos?

Por padrão, o browser bloqueia chamadas de `serveflow.vercel.app` para `serveflow.onrender.com` — são domínios diferentes (cross-origin). Sem configurar CORS, o frontend não consegue chamar a API em produção.

Exemplo pratico:

SEM CORS configurado:
Erro no browser: 'Access to XMLHttpRequest at onrender.com from origin vercel.app has been blocked by CORS policy'
COM CORS (CORS_ALLOWED_ORIGINS=https://serveflow.vercel.app):
Servidor responde com header:
Access-Control-Allow-Origin: https://serveflow.vercel.app
-> Browser permite a chamada
Em dev local: permite http://localhost:3000 e http://localhost:5173

4.8 @Async — Execucao Assincrona

O que é? Anotacao Spring que executa um metodo em uma thread separada do pool de threads, sem bloquear a thread principal que esta respondendo ao usuario.

Por que usamos?

Operacoes lentas (gravar logs no banco, enviar e-mail) nao devem atrasar a resposta ao usuario. Com @Async, essas operacoes rodam em paralelo — o usuario recebe o response imediatamente enquanto o trabalho pesado acontece em outra thread.

Exemplo pratico:

SEM @Async – usuario espera negocio + log:
OrderService.complete() -> 50ms (logica de negocio)
AuditService.logAction() -> +200ms (INSERT no banco de log)
Response ao usuario = 250ms <- usuario espera tudo
COM @Async – usuario espera apenas o negocio:
OrderService.complete() -> 50ms -> Response: 50ms (5x mais rapido)
AuditService.logAction() -> 200ms (em outra thread, em paralelo)

Onde encontrar: config/AsyncConfig.java + service/audit/AuditService.java

4.9 Spring Events — @EventListener

O que é: Mecanismo do Spring que permite publicar eventos e ter outros componentes reagindo sem se conhecerem diretamente — desacoplamento por eventos.

Por que usamos?

Sem eventos, OrderService teria que chamar diretamente CashierService.registerMovement() e KdsEventPublisher.notify() — acoplamento forte. Para adicionar notificacao por SMS seria necessario editar OrderService. Com eventos, OrderService apenas publica e qualquer componente pode escutar sem mudar o publicador.

Exemplo pratico:

```
// OrderService apenas publica – nao sabe quem vai escutar:
applicationEventPublisher.publishEvent(
new OrderCompletedEvent(order.getId(), order.getTotal())
);
// CashierEventListener reage de forma independente:
@EventListener @Async
public void onOrderCompleted(OrderCompletedEvent event) {
cashierService.registerMovement(event.getOrderId(), event.getTotal());
}
// Para adicionar SMS: cria novo listener, NAO TOCA em OrderService:
@EventListener
public void onOrderCompleted(OrderCompletedEvent event) {
smsService.notifyCustomer(event);
}
```

Onde encontrar: events/OrderCompletedEvent.java + service/cashier/CashierEventListener.java

4.10 @Scheduled — Tarefas Agendadas

O que é: Anotacao Spring que agenda execucao automatica de metodos em intervalos definidos por expressao cron — sem intervencao manual.

Por que usamos?

Manutencao periodica (limpeza de logs antigos, verificacao de alertas de estoque) nao deve ser disparada manualmente. @Scheduled garante execucao automatica dentro do proprio aplicativo, sem precisar de cron job externo.

Exemplo pratico:

```
@Scheduled(cron = "0 0 2 * * ?") // Todo dia as 02:00
public void cleanupOldLogs() {
// DELETE FROM access_log WHERE timestamp < NOW() - INTERVAL '90 days'
auditService.deleteLogsOlderThan(retentionDays);
}
Sem @Scheduled: tabela access_log cresceria indefinidamente.
Com @Scheduled: limpeza automatica toda noite, zero intervencao humana.
```

4.11 Padrao Repository

O que é: Padrão de design que abstrai o acesso a dados atrás de uma interface. O service usa a interface, sem saber qual tecnologia de banco está por baixo.

Por que usamos?

Sem o padrão Repository, o service importaria JpaRepository ou EntityManager diretamente — acoplamento entre lógica de negócio e infraestrutura. Impossível testar sem banco de dados real. Com Repository, podemos criar implementações in-memory para testes unitários.

Exemplo prático:

```
// Interface de domínio — sem dependência de framework:
public interface UserRepository {
    Optional<User> findByEmail(String email);
    User save(User user);
}

// Implementação com Spring Data JPA:
public interface SpringUserRepository extends JpaRepository<UserEntity, Long> {
    Optional<UserEntity> findByEmail(String email);
}

// Para testes — implementação in-memory, zero banco:
public class InMemoryUserRepository implements UserRepository {
    private Map<String, User> store = new HashMap<>();
    // UserService testado completamente sem PostgreSQL
}
```

4.12 DTO — Data Transfer Objects

O que é: Objetos simples usados para transferir dados entre camadas — separando o que entra (Input) do que sai (Output) e das entidades de banco.

Por que usamos?

Expor entidades JPA diretamente na API seria um problema: exporia campos internos (passwordHash, versão do Hibernate), dificultaria validação e acoplaria a API ao banco. DTOs definem contratos claros e independentes da implementação interna.

Exemplo prático:

```
// UserInput — o que o cliente ENVIA (com validação):
public record UserInput(
    @NotBlank String name,
    @Email @NotBlank String email,
    @NotBlank @Size(min=8) String password,
    @NotNull UserRole role
) {}

// UserOutput — o que o sistema RETORNA (sem campos sensíveis):
public record UserOutput(
    Long id, String name, String email, UserRole role, boolean active
) {}

// UserEntity — o que o banco ARMAZENA (NUNCA exposto na API):
String passwordHash; // nunca retornado
Integer version; // controle de concorrência do Hibernate
```

4.13 WebSocket + STOMP — Comunicacao em Tempo Real

O que é: WebSocket é um protocolo de comunicacao bidirecional e persistente. STOMP (Simple Text Oriented Messaging Protocol) adiciona conceitos de topicos, subscriptions e broker de mensagens sobre WebSocket.

Por que usamos?

HTTP é request/response — o cliente sempre inicia. Para o KDS receber novos pedidos instantaneamente e o caixa ver movimentos em tempo real, precisamos de server push: o servidor envia dados sem o cliente perguntar. Sem WebSocket seria necessario polling (requisicao a cada 2s) — ineficiente e com latencia alta.

Exemplo pratico:

```
SEM WebSocket (polling a cada 2s):
Sistema faz GET /kds/orders a cada 2 segundos
-> 30 requisicoes/minuto por tablet -> banco sobrecarregado
-> Cozinheiro ve pedido com ate 2s de atraso

COM WebSocket STOMP:
Garcom confirma pedido
-> OrderService publica OrderEvent
-> KdsEventPublisher envia para /topic/kds/orders
-> Todos os tablets conectados recebem em ~100ms automaticamente
-> Zero requisicoes de polling

Topicos do ServeFlow:
/topic/kds/orders -> atualizacao de status para cozinha
/topic/cashier/movements -> novos movimentos de caixa
/topic/cashier/sessions -> sessao aberta/fechada
```

5. Banco de Dados — Por que cada decisao?

5.1 PostgreSQL 15

O que é: Sistema de gerenciamento de banco de dados relacional open source, robusto e com suporte a tipos avançados (UUID nativo, ENUM, JSONB, arrays).

Por que usamos?

O ServeFlow tem relacionamentos complexos: pedido -> itens -> adicionais -> produtos -> ingredientes -> receitas -> estoque. Um banco relacional com JOINS e a escolha natural. PostgreSQL é melhor suportado pelo Supabase e tem tipos nativos (UUID, ENUM) usados no sistema.

Exemplo pratico:

```
-- Relatório financeiro cruzando 4 tabelas:
SELECT o.id, SUM(oi.total_price) as total, cs.opening_balance
FROM orders o
JOIN order_items oi ON oi.order_id = o.id
JOIN cash_movements cm ON cm.order_id = o.id
JOIN cash_sessions cs ON cs.id = cm.session_id
WHERE o.status = 'COMPLETED'
AND DATE(o.created_at) = CURRENT_DATE
GROUP BY o.id, cs.opening_balance;
```

5.2 Flyway — Versionamento do Banco

O que é: Ferramenta de migração de banco de dados que aplica scripts SQL versionados em ordem, uma única vez, garantindo consistência entre ambientes. É o 'Git do banco de dados'.

Por que usamos?

Sem Flyway, cada desenvolvedor teria um banco com schema diferente e o deploy em produção exigiria executar SQL manualmente — arriscado e difícil de rastrear. Flyway garante que dev, staging e produção sempre tenham exatamente a mesma estrutura.

Exemplo pratico:

```
Desenvolvedor cria: V27__add_table_number_to_orders.sql
-> ALTER TABLE orders ADD COLUMN table_number INTEGER;
Deploy em staging:
Flyway: 'V27 não aplicada -> executando...'
-> ALTER TABLE aplicado automaticamente
Deploy em produção (semana seguinte):
Flyway: 'V27 não aplicada -> executando...'
-> Mesma alteração, sem intervenção manual
-> Ambientes sempre sincronizados
ServeFlow: 26 migrations (V1 users+auth -> V26 cash_movements)
```

5.3 UUID como Primary Key

O que é: Identificador Universal Único — string de 36 caracteres gerada aleatoriamente, globalmente única. Alternativa a IDs sequenciais (1, 2, 3...).

Por que usamos?

IDs sequenciais expõe informações: um usuário pode tentar GET /orders/1, /orders/2... e enumerar todos os pedidos do sistema (IDOR — Insecure Direct Object Reference). UUIDs são imprevisíveis. Também permitem gerar IDs na aplicação antes do INSERT, sem depender de sequência do banco.

Exemplo prático:

```
ID sequencial: GET /api/orders/1, /api/orders/2, /api/orders/3
-> Atacante consegue listar todos os pedidos facilmente

UUID: GET /api/orders/550e8400-e29b-41d4-a716-446655440000
-> Próximo UUID impossível de prever
-> 2^122 possibilidades = 5.3 * 10^36 combinações

// Geração na aplicação (antes do INSERT):
order.setId(UUID.randomUUID());
orderRepository.save(order);
```

5.4 Supabase — PostgreSQL Gerenciado

O que é: Plataforma open source que provê PostgreSQL como serviço com Connection Pooling, Auth, Storage, Real-time e dashboard integrados.

Por que usamos?

Configurar PostgreSQL do zero exigiria: provisionar VPS, instalar PostgreSQL, configurar SSL, backups, monitoramento, pgBouncer, fail-over... Supabase oferece tudo isso no free tier com uma linha de connection string.

Recurso	Por que usamos?	Benefício prático
Connection Pooling (pgBouncer)	Cada conexão TCP ao PostgreSQL consome ~10MB RAM	10 conexões no pool servem 100 usuários simultâneos
SSL/TLS automático	Dados em trânsito criptografados	Nenhuma configuração manual de certificado
Backups automáticos	Recuperação de desastres	Restaurar banco em caso de falha crítica
PITR (Point-in-time Recovery)	Backups são snapshots; PITR restaura para qualquer segundo	Dev apaga tabela -> restaura estado de 1 minuto atrás
Row Level Security	Segurança a nível de linha no banco	Mesmo com bug no app, banco protege dados
Dashboard SQL Editor	Queries ad-hoc sem cliente externo	Gerente consulta dados pelo browser

6. Guia do Usuario — Como Usar o Sistema

Esta secao e destinada aos usuarios finais do ServeFlow. Aqui voce encontra, passo a passo, como realizar as operacoes do dia a dia de acordo com o seu perfil.

6.1 Primeiro Acesso — Login

1. Acesse o sistema pelo endereco fornecido pelo administrador
2. Informe seu **e-mail** e **senha** -> clique em **Entrar**
3. O sistema identifica seu perfil automaticamente e exhibe apenas as funcionalidades do seu role

Dica:

Esqueceu a senha? Clique em 'Esqueci minha senha' -> informe o e-mail -> aguarde o link na caixa de entrada. O link expira em minutos — verifique spam se nao chegar.

6.2 Perfil ADMIN — Administrador

Cadastrar usuario:

1. Menu **Usuarios** -> **Novo Usuario**
2. Preencha: nome, e-mail, senha inicial e perfil (ADMIN, GERENTE, GARCOM, COZINHEIRO, CAIXA)
3. Clique em **Salvar** -> comunique as credenciais ao colaborador

Alterar cargo: Usuarios -> colaborador -> **Alterar Cargo** -> selecionar -> confirmar

Desativar usuario: Usuarios -> colaborador -> **Desativar** -> perde acesso imediatamente

6.3 Perfil GERENTE — Gestao do Restaurante

Cadastrar produto:

1. **Produtos** -> **Novo Produto** -> nome, preco, tipo (PRODUTO ou INGREDIENTE)
2. Upload de imagem opcional (PNG/JPG, max. 8MB)
3. Marcar 'Requer ficha tecnica' se o produto consome ingredientes do estoque

Criar cardapio:

1. **Cardapios** -> **Novo Cardapio** -> nome + turno (ALMOCO, JANTAR, etc.)
2. Adicionar produtos -> preco personalizado por item (opcional)
3. Configurar agendamento automatico (opcional — ativa/desativa no horario definido)

Gerenciar estoque:

- **Entrada** (compra/reposicao): Estoque -> item -> Registrar Entrada -> quantidade + motivo
- **Perda** (vencimento/quebra): Estoque -> item -> Registrar Perda -> quantidade + motivo
- **Ajuste** (inventario fisico): Estoque -> item -> Ajuste -> nova quantidade real
- **Ficha Tecnica**: Receitas -> Nova Receita -> produto -> ingredientes + quantidades

Dica — Ficha Tecnica: Apos cadastrar a receita, cada venda deste produto deduz automaticamente os ingredientes do estoque. Exemplo: pizza de frango debita 200g de frango, 100g de queijo e 1 massa do estoque a cada pedido.

Modulo Financeiro:

- Conta a pagar/receber: Financeiro -> Contas -> Novo Lancamento -> descricao + valor + vencimento
- Liquidar: localizar conta -> Liquidar -> confirmar valor e data de pagamento
- Fluxo de caixa: Financeiro -> Fluxo de Caixa -> selecionar periodo

6.4 Perfil GARCOM — Registro de Pedidos

Registrar novo pedido:

1. Pedidos -> Novo Pedido -> tipo: MESA, BALCAO ou DELIVERY
2. MESA: informar numero da mesa | DELIVERY: informar CEP (endereco preenchido auto via ViaCEP)
3. Selecionar itens + quantidades + adicionais (ex: 'sem cebola', 'extra queijo')
4. Forma de pagamento -> Confirmar Pedido -> KDS recebe instantaneamente

Status	Significado	Acao do garcom
CRIADO	Pedido registrado	Aguardar confirmacao
CONFIRMADO	Na fila da cozinha	Aguardar preparo
PREPARANDO	Cozinha em preparo	Aguardar
PRONTO	Pedido pronto para retirada	Buscar na cozinha -> clicar Entregar
ENVIADO	Entregue ao cliente	Aguardar liquidacao no caixa
CONCLUIDO	Finalizado e pago	—

6.5 Perfil COZINHEIRO — KDS (Kitchen Display System)

A tela do KDS se atualiza automaticamente via WebSocket — sem necessidade de recarregar a pagina. Pedidos aparecem em ~100ms apos o garcom confirmar.

1. Pedido 'CONFIRMADO' aparece -> clicar Iniciar Preparo -> status: PREPARANDO
2. Preparo concluido -> clicar Pronto -> garcom e notificado para retirar
3. Item esgotado: Cardapio -> desativar disponibilidade -> garcom nao consegue mais pedir

Dica:

Mantenha o KDS em uma tela dedicada sempre visivel na cozinha. A atualizacao e automatica — nao precisa recarregar a pagina nunca.

6.6 Perfil CAIXA — Controle de Caixa

Abrir caixa (inicio do turno):

1. Caixa -> Abrir Caixa -> informar saldo inicial -> confirmar
2. So e possivel ter uma sessao aberta por vez — feche o turno anterior primeiro

Liquidar pedido (cliente pede a conta):

1. Caixa -> Pedidos Pendentes -> localizar pelo numero da mesa ou ID
2. Clicar Liquidar -> confirmar valor recebido e forma de pagamento

3. Sistema registra pagamento e fecha o pedido automaticamente

Movimentos manuais (sangria, troco, retirada):

Caixa -> Novo Movimento -> Entrada ou Saida -> valor + descricao

Movimentos de pedidos concluidos sao lancados automaticamente — o sistema cria o movimento de caixa via CashierEventListener quando um pedido e concluido.

Fechar caixa (fim do turno):

1. Caixa -> Fechar Caixa -> revisar resumo do turno
2. Informar saldo real contado em caixa
3. Se houver diferenca, ela e registrada automaticamente
4. Clicar Confirmar Fechamento — nenhum movimento pode ser adicionado apos isso

6.7 Resolucao de Problemas

Situacao	Solucao
Item nao aparece no cardapio	Produto deve estar ativo E item marcado como disponivel no cardapio
Pedido travado em CONFIRMADO	Cozinheiro deve clicar 'Iniciar Preparo' no KDS
Estoque caindo sem vendas manuais	Verificar ficha tecnica — pode haver desconto automatico por receita
Nao consigo abrir o caixa	Sessao anterior deve ser fechada primeiro (uma sessao por vez)
E-mail de reset nao chegou	Verificar spam; confirmar e-mail correto com administrador
KDS nao atualiza	Verificar conexao com internet — WebSocket requer conexao ativa

7. Infraestrutura e Deploy

7.1 Variaveis de Ambiente

Variavel	Descricao	Padrao (dev)
SPRING_PROFILES_ACTIVE	Perfil ativo	dev
DATABASE_URL	URL JDBC do PostgreSQL	—
JWT_SECRET	Chave HMAC-SHA256 (min. 256 bits)	obrigatorio
JWT_EXPIRATION	Expiracao do access token (ms)	900000 (15min)
CORS_ALLOWED_ORIGINS	Origins permitidas pelo CORS	http://localhost:3000
MAIL_HOST	Servidor SMTP	smtp.gmail.com
MAIL_PORT	Porta SMTP	587
MAIL_USERNAME	Usuario SMTP	obrigatorio
MAIL_PASSWORD	Senha SMTP	obrigatorio
MAIL_FROM	Remetente dos e-mails	obrigatorio
APP_BASE_URL	URL base para imagens	http://localhost:8080/api
AUDIT_RETENTION_DAYS	Retencao dos logs	90
SWAGGER_ENABLED	Ativa Swagger UI (false em prod)	true

7.2 Checklist de Deploy em Producao

- [] Todas as variaveis de ambiente configuradas no Render
- [] SPRING_PROFILES_ACTIVE=prod
- [] SWAGGER_ENABLED=false (nao expor API docs em producao)
- [] CORS_ALLOWED_ORIGINS aponta para o dominio Vercel correto
- [] Banco de dados Supabase acessivel pelo IP do Render
- [] Migrations Flyway executadas com sucesso (verificar logs de startup)
- [] Health check OK: GET /api/actuator/health -> 200
- [] Verificar logs de auditoria apos primeiro acesso

8. Boas Praticas e Manutencao

8.1 Adicionando Novo Modulo

Para adicionar um novo modulo (ex.: reservas de mesa), siga a estrutura:

1. V27__create_reservations.sql -> migration Flyway (tabela no banco)
2. ReservationEntity.java -> mapeamento JPA (@Entity, @Table)
3. SpringReservationRepository.java -> Spring Data JPA (JpaRepository)
4. Reservation.java -> dominio puro (sem frameworks)
5. ReservationRepository.java -> interface de dominio
6. ReservationInput/Output.java -> DTOs com Bean Validation
7. ReservationService.java -> logica de negocio (@Service)
8. ReservationController.java -> endpoint REST (@RestController)
9. ReservationNotFoundException.java -> excecao de dominio
10. GlobalExceptionHandler.java -> adicionar @ExceptionHandler

8.2 Seguranca

- Nunca commitar secrets no repositório — use variáveis de ambiente
- JWT_SECRET: mínimo 256 bits de entropia (gerar com: openssl rand -base64 32)
- SWAGGER_ENABLED=false em produção — não expor documentação da API
- CORS: listar origens explicitamente, nunca usar '*' em produção
- Refresh tokens são rotacionados a cada uso — invalidação automática se comprometidos

8.3 Auditoria e Monitoramento

Tabela	Conteúdo	Retenção padrão
access_log	IP, endpoint, método, status, timestamp de toda requisição	90 dias
audit_log	Usuário, ação (LOGIN, CREATE, UPDATE, DELETE), entidade, detalhe	90 dias
error_log	Endpoint, mensagem de erro, stack trace completo	90 dias

O AuditLogCleanupJob (@Scheduled) remove automaticamente registros mais antigos que AUDIT_RETENTION_DAYS dias — evitando crescimento indefinido das tabelas de log.

Referencia Rapida — Endpoints Principais

Endpoint	Método	Descrição	Perfil
/api/auth/login	POST	Login — retorna JWT + refreshToken	Público
/api/auth/refresh	POST	Renovar access token	Público
/api/auth/forgot-password	POST	Solicitar reset de senha por e-mail	Público
/api/auth/reset-password	POST	Confirmar nova senha com token	Público
/api/products	GET	Listar produtos ativos	Autenticado

Endpoint	Metodo	Descricao	Perfil
/api/products	POST	Cadastrar produto	ADMIN/GERENTE
/api/menus/active	GET	Cardapio ativo do turno atual	Autenticado
/api/orders	POST	Registrar pedido	GARCOM+
/api/orders/{id}/confirm	PATCH	Confirmar pedido	GARCOM+
/api/orders/{id}/complete	PATCH	Concluir pedido	CAIXA+
/api/kds/orders	GET	Fila de pedidos para cozinha	COZINHEIRO+
/api/kds/orders/{id}/prepare	PATCH	Iniciar preparo	COZINHEIRO+
/api/kds/orders/{id}/ready	PATCH	Marcar como pronto	COZINHEIRO+
/api/cashier/session/open	POST	Abrir sessao de caixa	CAIXA+
/api/cashier/session/{id}/close	POST	Fechar sessao de caixa	CAIXA+
/api/cashier/orders/{id}/settle	POST	Liquidar pedido no caixa	CAIXA+
/api/stock/report/consolidated	GET	Relatorio consolidado de estoque	GERENTE+
/api/dashboard/metrics	GET	Metricas gerais do restaurante	GERENTE+
/api/financial/cash-flow	GET	Fluxo de caixa por periodo	GERENTE+

ServeFlow — 207 classes Java | 26 migrations Flyway | 11 controllers | 14 services | 19 repositórios | Clean Architecture + DDD